

To simulate ccb scheduling Algorithm Priority Scheduling -

```
#include <stdio.h>
```

```
struct Process {
```

```
    int pid;
```

```
    int bt;
```

```
    int at;
```

```
    int pr, ct, tat, wt, rt, finished;
```

```
};
```

```
struct Gantt {
```

```
    int pid;
```

```
    int start;
```

```
    int end;
```

```
};
```

```
void print_gantt_chart (struct Gantt g[], int count) {
```

```
    printf ("\\n --- Gantt Chart ---\\n");
```

```
    printf (" ");
```

```
    for (int i = 0; i < count; i++) {
```

```
        for (int j = g[i].start; j < g[i].end; j++) printf (" ");
```

```
        printf (" ");
```

```
    }
```

```
    printf ("\\n |");
```

```
    for (int i = 0; i < count; i++) {
```

```
        int len = g[i].end - g[i].start;
```

```
        for (int j = 0; j < len - 1; j++) printf (" ");
```

```
        printf ("P%d", g[i].pid);
```

```
        for (int j = 0; j < len - 1; j++) printf (" ");
```

```
        printf ("|");
```

```
    }
```

```
    printf ("\\n");
```

```

for (int i = 0; i < count; i++) {
    for (int j = g[i].start; j < g[i].end; j++) printf("- ");
    printf(" ");
}
printf("\n");
printf("%d", g[0].start);
for (int i = 0; i < count; i++) {
    int len = g[i].end - g[i].start;
    for (int j = 0; j < len; j++) printf(" ");
    printf("%d", g[i].end);
}

```

printf("file")

```

void priority_non_preemptive(struct Prio pf[], int n) {
    int i, time = 10, completed = 0;
    struct Gantt g[100], ts[10];
    int gcount = 0;

```

```

while (completed < n) {
    int idx = -1;
    int highest = 9999;
    for (int i = 0; i < n; i++) {
        if (ts[i].pri < highest) {
            highest = ts[i].pri;
            idx = i;
        }
    }

```

```

    if (idx == -1)
        break;
    continue;
}

```

```

g[gcount].pid = p[idx].pid;
g[gcount].start = time;
time += p[idx].bt;
g[gcount].end = time;
gcount++;
p[idx].ct = time;
p[idx].tat = p[idx].ct - p[idx].at;
p[idx].wt = p[idx].tat - p[idx].bt;
p[idx].finished = 1;
completed++;

```

```

printf("\n --- Non-preemptive Priority Scheduling\n");
printf("PID \t AT \t BT \t PR \t CT \t TAT \t Wt \n");
for (int i=0; i<n; i++)

```

```

    printf("%d \t %d \t %d \t %d \t %d \t %d \t %d\n",
           p[i].pid, p[i].at, p[i].bt, p[i].pr,
           p[i].ct, p[i].tat, p[i].wt);

```

```

int avg-tat=0, avg-wt=0;

```

```

for (int i=0; i<n; i++)

```

```

    avg-tat += p[i].tat;

```

```

    avg-wt += p[i].wt;

```

```

printf("Avg TAT = %.2f\n", (float) avg-tat / n);

```

```

printf("Avg WT = %.2f\n", (float) avg-wt / n);

```

```

printf("gantt chart (g, gcount)");

```

```

void priority_preemptive (struct Process p[10], int n, int id) {
    int time = 0, completed = 0;
    struct Gantt g[100];
    int gcount = 0;
    int current = -1;
    while (completed < n) {
        int idx = -1;
        int highest = 9999;
        for (int i = 0; i < n; i++) {
            if (p[i].finished && p[i].at <= time) {
                if (p[i].pr < highest) {
                    highest = p[i].pr;
                    idx = i;
                }
            }
        }
        if (idx == -1) continue;
        if (current != idx) {
            if (current != -1) {
                g[gcount].end = time;
                g[gcount].start = time;
                g[gcount].pid = p[idx].pid;
                g[gcount].start = time;
                current = idx;
            }
            g[gcount].end = time;
            g[gcount].start = time;
            g[gcount].pid = p[idx].pid;
            g[gcount].start = time;
            current = idx;
        }
        time++;
        completed++;
    }
}

```

```

if (p[idx].rt == 0) {
    p[idx].ct = hmc;
    p[idx].tat = p[idx].ct - p[idx].at;
    p[idx].wt = p[idx].tat - p[idx].bt;
    p[idx].finished = 1;
    completed++;
    g[gcount].end = hmc;
    gcount++;
    current = -1;
}

```

```

}
printf("\n --- Preemptive Priority Scheduling --- \n");
printf("Pno \tat \tbt \tpr \tct \ttat \twat \n");
for (int i=0; i<n; i++) {
    printf("%d \t %d \t %d \t %d \t %d \t %d \t %d \n",
           p[i].pid, p[i].at, p[i].bt, p[i].pr,
           p[i].ct, p[i].tat, p[i].wt);
}

```

```

int avg_tat = 0, avg_wt = 0;
for (int i=0; i<n; i++) {
    avg_tat += p[i].tat;
    avg_wt += p[i].wt;
}

```

```

printf("Avg TAT = %0.2f \n", (float) avg_tat / n);
printf("Avg WT = %0.2f \n", (float) avg_wt / n);
print_gantt_chart(g, gcount);
}

```

```

int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    struct Process p[n];
    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("Enter AT, BT, Priority for P%d: ", i + 1);
        scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].pr);
        p[i].rt = p[i].bt;
        p[i].finished = 0;
    }

    struct Process p1[n], p2[n];
    for (int i = 0; i < n; i++) {
        p1[i] = p[i];
        p2[i] = p[i];
    }

    priority_non_preemptive(p1, n);
    priority_preemptive(p2, n);

    return 0;
}

```

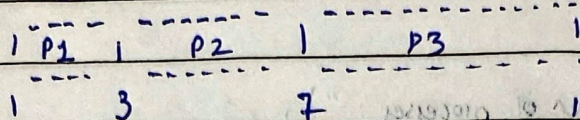
Enter number of processes: 3  
 Enter AT, BT, Priority for P1: 1 2 5  
 Enter AT, BT, Priority for P2: 2 4 3  
 Enter AT, BT, Priority for P3: 4 7 1

--- Non-preemptive Priority Scheduling ---

| PID | AT | BT | PR | CT | TAT | WT |
|-----|----|----|----|----|-----|----|
| 1   | 1  | 2  | 5  | 3  | 2   | 0  |
| 2   | 2  | 4  | 3  | 7  | 5   | 1  |
| 3   | 4  | 7  | 1  | 14 | 10  | 3  |

Avg TAT = 5.67      Avg WT = 1.33

# --- Gantt Chart ---

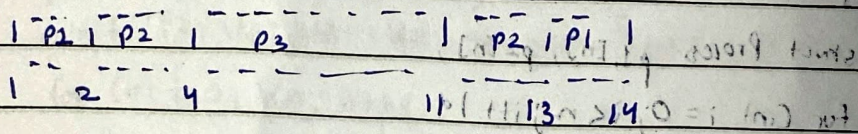


# --- Preemptive Priority scheduling ---

| PID | AT | BT | PR | CT | TAT | WT |
|-----|----|----|----|----|-----|----|
| 1   | 1  | 2  | 5  | 14 | 13  | 11 |
| 2   | 12 | 4  | 3  | 13 | 11  | 7  |
| 3   | 4  | 7  | 1  | 11 | 7   | 0  |

Avg TAT = 10.33 Avg WT = 6.00

# --- Gantt Chart ---



Process returned 0 (0x0) execution time 139.558s

| PID | AT | BT | PR | CT | TAT | WT |
|-----|----|----|----|----|-----|----|
| 0   | 1  | 1  | 2  | 5  | 1   | 1  |
| 1   | 2  | 1  | 3  | 1  | 1   | 1  |